

Project Ember — The AI HoneyToken Factory

A cybersecurity deception platform for African SMEs that uses Google Gemini to generate realistic decoy documents that trap ransomware and malicious insiders.

The Problem

African SMEs face the same threats as global enterprises — ransomware, insider data theft, and targeted attacks — but lack the budget for enterprise-grade deception tech (e.g., Illusive Networks, Attivo). When ransomware hits, it's often too late: files are encrypted, data is exfiltrated, and there are no logs to trace the source.

How Project Ember Solves It

Project Ember flips the asymmetry: instead of trying to block every attack, you **plant decoys** that look like high-value targets (payroll spreadsheets, HR records, IT credentials) across your file servers. When an attacker touches one — whether it's ransomware encrypting files, or an insider browsing sensitive docs — the dashboard alerts you in real time with the attacker's IP address and browser fingerprint.

The Flow

1. **DEPLOY** —> Gemini generates 3 realistic decoy files (.txt, .md, .html) with hidden tracking pixels and fake corporate data (payrolls, employee records, IT audit reports)
 2. **PLACE** —> Copy the files to your file server, shared drive, or sensitive folder where attackers would look
 3. **WAIT** —> The dashboard monitors:
 - Tracking pixel hits (file opened / viewed via browser)
 - Filesystem changes (modified / deleted / renamed)
 - Attacker IP address + User-Agent extracted from beacon
 - All via the watchdog daemon + tracking server
 4. **ALERT** —> When a trap is triggered (within 1 second):
 - Full-screen red banner persists for 10 seconds
 - Shows file name, timestamp, attacker IP, browser profile
 - Sky News–style audible alarm
 - Graph node turns red with pulsing animation
 - Event counter updates in real time
-

Current Capabilities

Feature	Status
AI Decoy Generation — Gemini 2.0 Flash generates convincing corporate documents with African context (KES/NGN currencies, KRA/FIRS compliance, local names, East African logistics routes)	✓
4 Department Templates — Finance (payroll), HR (employee records), IT (network audit), Operations (supply chain)	✓
3 Output Formats — .txt, .md, .html per deployment	✓
Hidden Tracking Pixels — beacon in HTML/MD files; logs hits to the dashboard	✓
Attacker Intelligence — IP address + User-Agent extracted from every beacon request	✓
Filesystem Watchdog — detects MODIFIED, DELETED, MOVED, CREATED events in real time	✓
Network Topology Graph — tree-layout visualization (server → departments → decoy files) using networkx + matplotlib; nodes flash red on access	✓
Live Threat Monitor — event log with timestamps, types, file names, IPs, and User-Agents	✓
Explosive Alert — full-width red banner at page top with file details + attacker intel, persists 10s	✓
Flashing Red Nodes — graph nodes pulse red automatically when a file is accessed	✓
Audible Alarm — Sky News–style two-tone alert (bass thump + rising sweep) via Web Audio API	✓
1-Second Polling — background poller checks every 1s for instant graph + alert updates without page flicker	✓
Bootstrap Icons — professional UI, clean aesthetics	✓
Mock Mode — works without API key for UI testing (shows placeholder text)	✓

Tech Stack

Layer	Technology
AI Engine	Google Gemini 2.0 Flash via google-genai SDK
Dashboard	Streamlit 1.60+
Graph Visualization	networkx + matplotlib
Fake Data	Faker (custom African name pool, IBANs, ID numbers)
File Monitoring	watchdog (filesystem event listener)
Tracking Server	Python http.server (lightweight, returns 1×1 GIF pixel, logs HTTP request metadata)
Alert Sound	Web Audio API (generated in-browser, no audio files)
Icons	Bootstrap Icons CDN
Background Tasks	Python threading + Streamlit fragments

How to Use

1. Install

```
cd Project_EMBER
pip install -r requirements.txt
```

```
export GEMINI_API_KEY="your_google_ai_key_here"
```

2. Launch

```
python3 run.py
```

This starts three services:

- **Tracking server** on **:8765** — handles beacon pixel requests, logs IP + User-Agent
- **Watchdog daemon** — monitors **./honeytokens/** for file changes
- **Streamlit dashboard** on **:8501** — the UI

3. Deploy Decoys

1. Open **http://localhost:8501** in a browser
2. Enter a **Company Name** (e.g., "Acme Kenya Ltd")
3. Pick a **Department** (Finance / HR / IT / Operations)
4. Click **Deploy HoneyTokens**

Three files appear in **./honeytokens/**:

- **Company_Dept_20260724.html** — open this in a browser to test tracking
- **Company_Dept_20260724.md**
- **Company_Dept_20260724.txt**

4. Place the Traps

Copy the generated files to your target environment — a file server, a shared drive, an HR document folder, or anywhere an attacker might browse.

5. Monitor

Keep the dashboard open. When someone (or something) touches a decoy:

Action	What the dashboard shows
Opens .html in a browser	Tracking pixel fires → red banner + sound + graph node turns red + logs attacker IP + browser
Modifies a file (ransomware)	Watchdog catches → same alert + event log entry
Deletes/moves a file	Watchdog catches → alert + log

Demo: Track an Attacker in Real Time

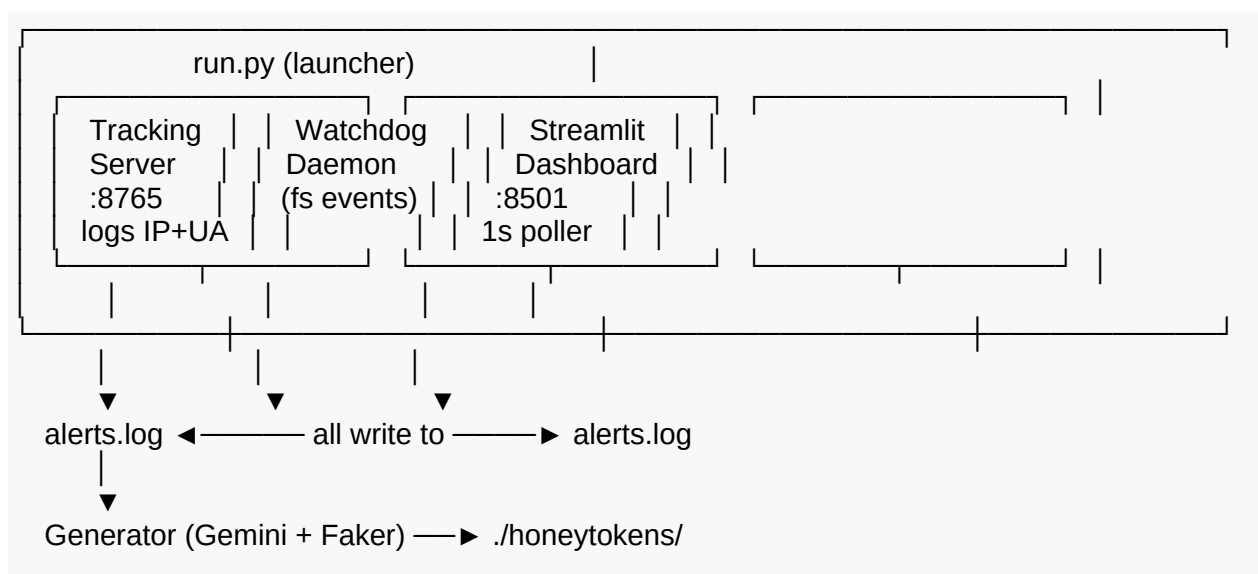
```
# Simulate an attacker opening a stolen file from a remote machine
curl -A "Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120" \
```

```
"http://localhost:8765/track?file=payroll_2026_q3.html"
```

Within 1 second, the dashboard shows:

- **Red banner:** "CRITICAL INTRUSION DETECTED — payroll_2026_q3.html"
- **Attacker IP:** 127.0.0.1 (or the actual remote IP)
- **Browser:** Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120
- **Sound:** Sky News-style alert
- **Graph:** Node flashes red

Architecture



File Structure

```
Project_EMBER/
├── app.py           # Streamlit dashboard (graph, alerts, metrics, 1s poller)
├── generator.py     # Gemini + Faker decoy generation
├── watchdog_daemon.py # Filesystem event monitor
├── run.py          # Launcher (tracking + watchdog + streamlit)
├── requirements.txt # Python dependencies
├── README.md       # This file
├── honeytokens/    # Generated decoy files
│   ├── manifest.json # Tracking manifest
│   ├── *.html        # Decoy with pixel beacon
│   ├── *.md
│   └── *.txt
```

Deploying for Real

For production use:

- **Reverse proxy** (nginx/Caddy) to serve both Streamlit and tracking on port 80/443
- **Persistent alerts.log** with rotation
- **Watchdog** on the actual file server path (not local `./honeytokens`)
- **SSL** for all endpoints
- **Email/Slack/Webhook** integration for off-hours alerts
- **GeoIP lookup** on attacker IP addresses for physical location tracking